

Level-Up Logics

Leveraging Three Game Design Platforms to Teach Coding

Topher Maraffi

North Carolina State University, Media Arts, Design and Technology, Raleigh NC
cmaraff@ncsu.edu

ABSTRACT

Coding has become an integral tool for designers, but the logical structures of programming can be intimidating for many students in the arts. To make learning the fundamentals of coding less stressful and more appealing, I revamped a Creative Coding elective in our *Media Arts, Design and Technology* (MADTech) program to progress students through three increasingly technical game design platforms. Students were taught how to apply the player-centric MDA (Mechanics, Dynamics, Aesthetics) game design framework to Fortnite Creative, a popular 3D platform that doesn't require any coding but instead has students configure devices in the game world to implement logical interaction. Students used a library of game assets to build a complex prototype that required collectibles to open a door and progress in the game. These core game mechanics were carried through to similar projects in Unreal Engine and Unity, while the MDA framework enabled students to creatively vary aesthetics and dynamics to make each game look and feel unique. By applying the same logical structures in three platforms, progressing from graphical node-based Blueprints to C# scripting, students gained a better grasp of fundamental coding concepts and showed a preference for scripting by the end of the semester. Though this curriculum was primarily targeted at students in the College of Design, students from more technical backgrounds like Computer Science were still challenged by the more aesthetic features of this design-centric approach to game development.

CCS CONCEPTS

• Fundamentals; • Applied computing; • Education; Computer games; • Media arts;

KEYWORDS

Coding Fundamentals, Game Design, Visual Programming, Interaction Logics

ACM Reference Format:

Topher Maraffi. 2024. Level-Up Logics: Leveraging Three Game Design Platforms to Teach Coding. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH Educators Forum '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641235.3664434>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH Educators Forum '24, July 27–August 01, 2024, Denver, CO, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0517-5/24/07

<https://doi.org/10.1145/3641235.3664434>

1 INTRODUCTION

With the video game industry continuing to dominate other media like film and streaming in 2023, and game engines being increasingly used for non-gaming applications like virtual production and spatial computing, it is becoming essential that media art students become more technical and add coding to their skillsets. However, many students from an arts background are intimidated by the abstract logical concepts and related syntax of a programming language like C#. In fall 2023, I re-designed the curriculum of our Creative Coding course to progressively teach students the operational logics of game design through three coding platforms: Fortnite Creative devices, Unreal Engine blueprints, and Unity C# scripting. This allowed me to front-load the course with game design concepts like the MDA (Mechanics, Dynamics, Aesthetics) framework [Hunicke, 2004] and Magic Circle [Stenros, 2014], without getting bogged down by also having students learn scripting syntax. Instead, students focused on the player-centric design of game aesthetics and dynamics in Fortnite Creative, and only had to configure 3D devices in the game to create complex interactions and supporting mechanics. Students were required to build a level with core dynamics that challenged the player to open a locked door by finding a set amount of collectible items, such as keys or coins, which would lead to a win condition for the level. Coding concepts like conditional statements, loops, and variables were built into the device settings, so that students began to understand how player interaction was coded. Students were also required to research some other secondary challenge or mini-game to incorporate into their core dynamics. At the end of the project, students play-tested several of their peer games and gave feedback according to the game design concepts learned.

While the first project allowed the students to create complex 3D games with minimal technical skills, the design was constrained by the Fortnite Creative affordances. Students couldn't use custom sounds, images, or 3D game objects, so all the levels had the aesthetics of Fortnite. The second project in Unreal Engine 5 did not have any creative constraints, but students had to build more of the components from scratch in the game engine. I encouraged them to use available asset libraries to rapidly prototype their maps, such as Quixel Bridge Megascans and Metahumans, or download animations from the Epic Game Marketplace and Mixamo. Students again applied the MDA and other game design concepts to their projects, and learned how to develop core dynamics that were similar to their Fortnite gameplay, but the logical structures previously obscured in the Fortnite devices were visible as branching nodes in the UE5 blueprint graphs. Though the resulting third-person games were not as complex as the Fortnite games, they were more creative and many students went beyond the core dynamics. The third project

Table 1: Metadata in tabular format.

Summary	Students learn to code interaction in three progressive game platforms.
Learning Outcomes	Students should be able to creatively code and discuss logics that support dynamic game play.
Classification(s)	Fundamentals, Computer games, Media Arts.
Audience	Undergraduate and graduate art students in the School of Design, and some CS students.
Dependencies	Free software available in our lab.
Prerequisites	No prerequisites or previous coding experience was required.
Strengths	Repeating similar dynamics helped them to learn the foundations of 3D game coding.
Weaknesses	If a student didn't keep up with the assignments, the course became progressively harder.
Variants	They were able to be creative within the context of the project requirements.
Assessment	The students presented demos of playable prototypes and then designed a research poster on the process.

introduced students to C# scripting in Unity 2022 game engine using Microsoft Visual Studio Community Edition. Students started the game design process by customizing sample scripts that created some core dynamics, this time as a first-person game, and then were required to create new C# scripts to develop unique dynamics to their game. Students were intimidated at first, but then their previous experience helped them to complete a playable prototype in a few weeks. In each project, students presented their prototypes in a video demo where they explained their game design and development process using industry standard terms and concepts.

2 COURSE MATERIALS

Excerpts from student projects in our Creative Coding 2023 Demo Reel (class tutorials are also available on my YouTube channel): [https://youtu.be/6WZhixNhIh4?si=\\$7IYY6_-lLOKiOFi8](https://youtu.be/6WZhixNhIh4?si=$7IYY6_-lLOKiOFi8) Example of a student poster: <https://tophermaraffi.com/wp-content/uploads/2024/02/molliegamesposter.jpg> Please contact me for the course syllabus and sample C# scripts.

3 RESULTS AND DISCUSSION

The final project for students was to finish either their Unreal Engine or Unity prototypes, build the level into a playable game, and then record a video demo of their design process. Surprisingly and without exception, all students chose to continue C# scripting in Unity game engine. This was unexpected because many students had liked using blueprints in Unreal Engine and were initially hesitant about learning to script in Unity. In their final posters where they analyzed all three games, some of the students explained that they preferred C# scripting because it seemed like they could do more: “Throughout working with all three gaming platforms, I have found that C# scripting in Unity is the most challenging, but ultimately allows for a wide range of complex game mechanics and interactions between different mechanics - Stella” and “I continued refining my project in Unity because C# scripting gave me the most freedom to develop my game the way I envisioned it - Mollie”. Others chose Unity because it ran better on their computer, “I ultimately chose Unity, because of the freedom of the code and the accessibility of the program allowing me to work on my simple laptop - Maleah”.

By the end of the course, most students felt more confident in their technical abilities and appreciated the ramp-up to full scripting: “Learning the concepts of variables, triggers, colliders, and conditions with Unreal and Fortnite before trying C# was helpful – Emily”. This methodology is consistent with computer science studies that have shown that graphical game programming is “more feasible, straightforward and incorporate a much lower learning threshold” than immediately scripting in a program like Unity [Comber, 2019], and motivates students because Fortnite is a popular 3D gaming platform. Where a CS approach may use any game engine as motivation to learn coding concepts [Serrano-Laguna, 2014], it is important for my design-centric approach to use industry standard platforms that balance coding with game aesthetics. By exposing our MADTech students to three popular engines in one semester, they are better prepared to build on those foundations in our upper-level design courses that use Unity and Unreal Engine. It was also helpful having a mix of undergraduate and graduate students from the College of Design, along with a few students from other disciplines like CS and communications, because they encouraged and assisted each other through technical and aesthetic challenges.

REFERENCES

- Comber, O., Motschnig, R., Mayer, H. and Haselberger, D., 2019, April. Engaging students in computer science education through game development with unity. In 2019 IEEE Global Engineering Education Conference (Educon) (pp. 199-205). IEEE.
- Hunicke, R., LeBlanc, M. and Zubeck, R., 2004, July. MDA: A formal approach to game design and game research. In *Proceedings of the AAAI Workshop on Challenges in Game AI* (Vol. 4, No. 1, p. 1722). <https://www.game-developers.org/wp-content/uploads/2022/09/MDA.pdf>
- Serrano-Laguna, Á., Torrente, J., Manero, B. and Fernandez-Manjon, B., 2014, October. A game engine to learn computer science languages. In 2014 IEEE Frontiers in Education Conference (FIE) Proceedings (pp. 1-8). IEEE.
- Stenos, J., 2014. In defence of a magic circle: the social, mental and cultural boundaries of play. *Transactions of the digital games research association*, 1(2). <http://todigra.org/index.php/todigra/article/viewFile/10/27>